ENTITY-LIFE MODELING AND ADA 9X

Jeffrey R. Carter and Bo I. Sanden

1. Introduction

Sanden presents Entity-Life Modeling (ELM), which combines object- and process-based concepts to identify problem entities which are modeled as active subjects and passive objects in reactive software [1]. Because the resulting design incorporates information hiding, multiple concurrent threads of control, and resource sharing, Sanden used Ada 83 to demonstrate the implementation of ELM designs, using Ada's tasks and packages.

Ada 83 has a number of shortcomings for reactive systems. Exclusive access to shared resources requires additional tasks, which add overhead to the solution not required by the design. Interrupt handling is also provided by tasks, which may increase interrupt latency unacceptably. Asynchronous transfer of control, in which processing is aborted in response to an event, is another concept that Ada 83 does not handle well.

The proposed revision to Ada, called Ada 9X, provides a number of features intended to address these shortcomings. This paper examines the effect of these features on the implementation of ELM designs. Protected units, protected procedure interrupt handlers, priority entry queuing, dynamic task priorities, and asynchronous select statements are shown to simplify the implementation of ELM designs when compared to Ada 83 implementations.

There are no further explicit references to Sanden [1] in this paper. The entire paper may be considered such a reference.

2. Information Hiding and Resource Sharing in Ada 9X

ELM emphasizes information hiding as a design principle, and recommends implementing it by information-hiding packages and Abstract Data Types (ADTs). An information-hiding package is a single information-hiding object, while an ADT is a type, and a variable of the type is the information-hiding object. ELM, being primarily concerned with reactive, concurrent systems, also considers the use of information-hiding packages and ADTs in a concurrent environment; the complication of a concurrent environment is to provide tasks with exclusive access to shared resources. ELM provides exclusive access by requiring tasks which use a resource to first rendezvous with a guardian task.

Using guardian tasks introduces additional threads of control into the system, and uses a single construct, tasks, for two different purposes, implementing subjects and guaranteeing exclusive access to shared resources. Ada 9X introduces the protected unit to replace guardian tasks. Protected units do not introduce additional threads of control. Replacing guardian tasks with protected units significantly reduces the processor load in concurrent systems implemented on a single processor, without significantly changing the software architecture or existing code [2]. Protected units do not eliminate the need for order rules and other techniques to avoid deadlock.

Protected units are variables of protected types, similar to ADTs. They are invoked using a syntax similar to information-hiding packages, which simplifies a system's software architecture and design [3]. They therefore combine the advantages of both information-hiding packages and ADTs into a single structure.

Ada 9X allows information-hiding packages and ADTs to be implemented in the same way as Ada 83. Protected units offer advantages over both of these traditional Ada-83 ways of implementing shared information-hiding resources. While a protected unit could simply replace the guardian task in the body of an information-hiding package, exporting a protected type makes modification easier should multiple instances of the object be required.

ADTs are implemented using private types. The privacy of such types cannot be compromised using normal Ada-83 language features. Ada 9X' child-package feature can be used to compromise the privacy of private types, thereby significantly reducing the safety and reliability of a system. For example, a child package could be used to access the full definition of a private type directly, bypassing the safeguards implemented by the package designer and possibly corrupting the state of the system. If the type is implemented as a special-purpose guardian task type, a child package could be used to abort the task.

The safety and privacy of protected units cannot be compromised, regardless of whether the protected type is visible or the full definition of a private type. Protected units do not contain a thread of control, so abort does not apply to them.

As in Ada 83, Ada 9X emphasizes the use of special-purpose protected units over general-purpose semaphores. It is possible to use a protected type to define a semaphore (see below). However, there are a number of arguments for using special-purpose guardian tasks instead of semaphores, including safety, portability, and run-time efficiency [4]. These arguments also apply to protected units.

2.1. Protected-Unit Syntax

Protected units combine the syntax of tasks and records. As with tasks, one can declare a single protected object or a protected type:

```
protected [type] name [(discriminants)] is
   {subprogram or entry declaration}
[  private
   {subprogram, entry, or component declaration}]
end name;

protected body name is
   {subprogram declaration, subprogram body, or entry body}
end name;
```

Only a protected type may have discriminants. Ada 9X allows discriminants on tasks and protected units. Such discriminants replace initialization entries for task types, and similarly provide initialization for variables of protected types. Examples of using discriminants are given below.

As an example of a protected unit, a semaphore may be implemented as

```
protected type Semaphore is
   entry Request;
   entry Release;
private -- Semaphore
   Acquired : Boolean := False;
end Semaphore;

protected body Semaphore is
   entry Request when not Acquired is
      -- No declarations
   begin -- Request
      Acquired := True;
   end Request;
```

```
   entry Release when Acquired is
      -- No declarations
   begin -- Release
      Acquired := False;
   end Release;
end Semaphore;

Protection : Semaphore;
...
Protection.Request;
-- use resource
Protection.Release;
```

Protected units can contain functions, procedures, and entries, each of which provides a different kind of access to the unit. A protected unit has an "execution resource" associated with it, which is similar to a semaphore or lock. Protected functions provide read-only access to the unit. Calls to a protected function may proceed concurrently with other function calls, but not with procedure or entry calls, and may not modify any of the unit's components. Protected procedures and entries provide exclusive read-write access to the unit; entries provide for call queuing and procedures do not. Calls to a procedure must first acquire the unit's execution resource before executing, providing exclusive access. Calls to an entry first acquire the unit's execution resource, then evaluate the entry body's guard (between `when` and `is`). If the guard is true, the entry body may be executed. Otherwise, the calling task is queued on the entry and the unit's execution resource is released.

Protected units have the potential to simplify libraries of reusable components. In Ada 83, a reusable component intended for use in a concurrent system must contain a task, and operations of the component must rendezvous with the task. The run-time effects of activating a task and rendezvousing with it for every operation are significant enough that an otherwise sequential system would not want to reuse a component which contains a task. This leads to multiple implementations of a single component for concurrent and sequential systems. Since acquiring a protected unit's execution resource is a simple operation, protected types may allow a single implementation of a component to be used by all systems [5].

2.2. Protected Units in the RTS Problem

Sanden presents two guardian tasks in the ELM solution to the Remote Temperature Sensor problem which may be replaced by protected units, `Control_Interval` and `Thermometer`. (Task `Output` is an agent, which takes independent action at a subject's request. Agents should be implemented as tasks.) `Control_Interval` and `Thermometer` may be implemented as protected units. Using `Control_Interval` as an example yields:

```
type Interval_Set is array (Furnace_Id) of Duration;

protected Control_Interval is
   procedure Change (Furnace : in Furnace_Id; New_Value : in Duration);
   function Get (Furnace : Furnace_Id) return Duration;
private -- Control_Interval
   Interval : Interval_Set := Interval_Set'(others => Invalid_Interval);
end Control_Interval;

protected body Control_Interval is
   procedure Change (Furnace : in Furnace_Id; New_Value : in Duration) is
      -- No declarations
   begin -- Change
      Interval (Furnace) := New_Value;
   end Change;
```

```
    function Get (Furnace : Furnace_Id) return Duration is
       -- No declarations
    begin -- Get
       return Interval (Furnace);
    end Get;
end Control_Interval;
```

The reader may want to generalize `Control_Interval` into a reusable `Shared_Array` component.

3. Interrupt Handling in Ada 9X

The reactive systems that interest ELM would normally be implemented using compilers validated against the Systems Programming and Real-Time Systems Annexes (Annexes G and H) of the 9X ARM [6] (validation against Annex H implies validation against Annex G). Annex G provides interrupt handling. Ada 83 attaches an interrupt to a task entry by means of a `System.Address`. Ada 9X attaches interrupts to procedures of protected units by means of an `Ada.Interrupts.Interrupt_Id`:

```
with Ada.Interrupts;
use Ada;
...
Id : constant Interrupts.Interrupt_Id := ...;
...
protected Interrupt_Handler is
   procedure Handler;
   pragma Attach_Handler (Handler, Id);
private -- Interrupt_Handler
   ...
end Interrupt_Handler;
```

`Ada.Interrupts.Interrupt_Id` is a discrete type. We will assume that it is an integer type. Annex G also allows for interrupt handlers to be attached dynamically.

3.1. Interrupt Handling in the RTS Problem

`Thermometer` demonstrates the use of entries and requeue statements in protected units, as well as the form of interrupt handlers in Ada 9X. Requeue statements allow an entry call to be queued on another entry for later processing.

```
type State_Id is (Ready, Reading, Temperature_Available);

protected Thermometer is
   entry Get (Furnace : in Furnace_Id; Temperature : out Temperature_Value);
private -- Thermometer
   procedure Handler;
   pragma Attach_Handler (Handler, 64);

   entry Finish (Furnace : in Furnace_Id; Temperature : out Temperature_Value);

   Physical_Temperature : Temperature_Value;
   for Physical_Temperature'Address use 16#1234_5678#;

   State                : State_Id := Ready;
end Thermometer;

protected body Thermometer is
   entry Get (Furnace : in Furnace_Id; Temperature : out Temperature_Value)
      when State = Ready
   is
      -- declarations
```

```
        begin -- Get
            State := Reading;
            -- Issue command to physical thermometer. Wait for Interrupt.
            requeue Finish;
        end Get;

        procedure Handler is
            -- No declarations
        begin -- Handler
            if State = Reading then
                State := Temperature_Available;
            end if;
        end Handler;

        entry Finish (Furnace : in Furnace_Id; Temperature : out Temperature_Value)
            when State = Temperature_Available
        is
            -- No declarations
        begin -- Finish
            Temperature := Physical_Temperature;
            State := Ready;
        end Finish;
    end Thermometer;
```

The requeue in entry body `Get` means the caller remains queued on entry `Finish` until `State` becomes `Temperature_Available`. The caller remains suspended until entry body `Finish` has completed.

This code assumes that interrupt 64 only occurs when the physical thermometer is triggered by the software, which is only done in entry `Get`. The physical thermometer stores the temperature at `Physical_Temperature`'s address before generating the interrupt.

Since `State` is initially equal to `Ready`, the guard on `Get` is true and it may be executed. `Get` triggers the physical thermometer to read the temperature and generate the interrupt, sets `State` to `Reading`, and queues the call on entry `Finish`. Both entries are now blocked until the interrupt occurs, which sets `State` to `Temperature_Available`. The guard on `Finish` is now true; since there is a call queued on it because of the requeue in `Get`, `Finish` will now execute, returning the temperature to the calling task and setting `State` back to `Ready`, allowing another call to `Get` to execute.

4. Periodic Tasks in Ada 9X

Reactive systems frequently contain many periodic tasks, such as samplers, regulators, and output generators. In Ada 83, such tasks reschedule themselves by executing a delay statement. To avoid cumulative drift, the delay interval is calculated from the time of the next execution and the current time:

```
Next := Next + Interval;
delay Next - Calendar.Clock;
```

There are problems with this approach. The time returned by `Clock` may be inaccurate, either because time has passed since `Clock` sampled the time, or because of time changes such as starting or ending daylight-saving time. Even if `Clock` returns the correct time, the subtraction operation requires some time to execute. Although this time is small, it may be significant for some time-critical applications.

Ada 9X avoids these problems by introducing the delay until statement, which takes a time instead of a duration. Annex H includes the predefined package `Ada.Real_Time`, which can be used instead of `Calendar` for a high-resolution, monotonic clock. Annex H also

requires that the compiler vendor document the accuracy of delay and delay until statements. These features change a delay loop into:

```
Next : Ada.Real_Time.Time := Ada.Real_Time.Clock;
...
Forever : loop
   Action;
   Next := Next + Interval;
   delay until Next;
end loop Forever;
```

4.1. Delay Until in the RTS Problem

Sanden's solution to the RTS problem includes a periodic task type which periodically samples a furnace's temperature. We can modify this task to take advantage of delay until statements and task discriminants:

```
with Ada.Real_Time;
use Ada;
package body Remote is
   ...
   task type Furnace_Sampler (Id : Furnace_Id) is
   -- Periodically sample a furnace's temperature
      entry Alert; -- Alert the task that its sampling interval has changed
   end Furnace_Sampler;

   type Furnace_Ptr is access Furnace_Sampler;
   type Furnace_Set is array (Furnace_Id) of Furnace_Ptr;

   Furnace : Furnace_Set;

   task body Furnace_Sampler is
      Temperature : Temperature_Value;
      Next        : Real_Time.Time;
      Interval    : Real_Time.Time_Span;
   begin -- Furnace_Sampler
      accept Alert; -- Wait for valid sampling interval
      Next := Real_Time.Clock;

      Forever : loop
         Thermometer.Get (Furnace => Id, Temperature => Temperature);
         Interval := Real_Time.To_Time_Span (Control_Interval.Get (Id) );
         Next := Next + Interval;

         select
            Output.Send (Furnace => Id, Temperature => Temperature);
         or
            delay until Next;
         end select;

         select
            accept Alert; -- check for change to Interval
            Next := Real_Time.clock;
         or
            delay until Next;
         end select;
      end loop Forever;
   end Furnace_Sampler;
   ...
begin -- Remote
   Activate : for F in Furnace'Range loop
      Furnace (F) := new Furnace_Sampler'(Id => F);
```

```
    end loop Activate;
end Remote;
```

In Ada 83, multiple instances of a task type could only obtain initialization information by a rendezvous. This imposed serialization on otherwise-parallel systems. Task discriminants allow this information to be provided when the task object is declared. An additional advantage is that entry `Alert` now has a single purpose. Sanden used `Alert` for two purposes, telling the task its identifier and telling the task that its sampling interval has changed.

Because tasks are limited types, the array of tasks cannot be initialized when declared. Instead, an access type which designates the task type must be introduced, and the array of tasks changes to an array of access values. The tasks are dynamically allocated, allowing their identifiers to be provided during allocation.

5. Priorities and Queuing Policies in Ada 9X

Annexes G and H provide system-defined task identifiers, which may be used to dynamically set the priorities of tasks. They also provide pragma `Queuing_Policy`, which specifies the order of calls in entry queues. One possible queuing policy must be `Priority_Queuing`, which causes calls in entry queues to be ordered based on the caller's priority. If a caller's priority changes, its position in the queue also changes to reflect its new priority. At least thirty priority levels are available.

Task identifiers are provided by the private type `Ada.Task_Identification.Task_Id`. Dynamic priorities may be set by calling `Ada.Dynamic_Priorities.Set_Priority` with the desired priority and task identifier.

Dynamic priorities and priority queuing can simplify Sanden's Flexible Manufacturing System solution. Priority queuing of jobs for workstations and dynamic modification of jobs' priorities make up a significant part of package `Workstations`. This is demonstrated in Section 7.

6. Asynchronous Transfer of Control in Ada 9X

Ada 83 does not provide an easy solution to problems which require transferring control out of a sequence of statements when an event occurs. A common example is a mathematical calculation which repeatedly improves an approximate result until it converges to an accurate value. If the calculation does not converge within some time limit, a reactive system may need to terminate the calculation and use the approximate value.

Ada 9X provides an Asynchronous Transfer of Control (ATC) version of the select statement for such cases:

```
select
   trigger
   [sequence of statements]
then abort
   abortable sequence of statements
end select;
```

The trigger is a delay statement or entry call. If the delay has expired or the entry call can be accepted immediately, the optional sequence of statements is executed and control passes to the next statement after the select statement. Otherwise, the abortable sequence of statements begins executing. If the triggering event occurs before the abortable sequence of statements completes, the abortable sequence of statements is aborted and the optional sequence of statements after the trigger is executed as above. Otherwise, the trigger is canceled and the next statement after the select statement is executed.

The mathematical calculation example would be implemented as:

```
protected Result is
   function Get return Real;
   procedure Set (Update : in Real);
private -- Result
   Value : Real;
end Result;
Epsilon : constant Real := ...;
...
Approximation := Initial_Estimate;
Previous := Approximation + 2.0 * Epsilon;
select
   delay 0.1;
then abort
   Refine : loop
      exit Refine when abs (Approximation - Previous) < Epsilon;
      Previous := Approximation;
      -- improve accuracy of Approximation
      Result.Set (Update => Approximation);
   end loop Refine;
end select;
Value := Result.Get;
```

In this example, the system has one tenth of a second to calculate the result; if the result cannot be calculated by then, the calculation will be aborted and the latest approximation used.

7. The Flexible Manufacturing System

Sanden presents the moderately-complex Flexible Manufacturing System (FMS) problem and a fairly-complete, Ada-83 solution to it. We will use Sanden's design but implement important aspects of the solution using the Ada-9X features discussed above. Besides replacing guardian tasks with protected units, we will eliminate the explicit priority queuing for jobs by using priority entry queuing and dynamic priorities. We will also significantly improve interrupt handling by providing an interrupt handler for each Automated Guided Vehicle (AGV) and each workstation. Sanden used one handler for all AGVs and one for all workstations, since Ada 83 does not provide a way to give each instance of a task type its own interrupt address.

7.1. Design

Figure 1 shows the overall software design of the FMS. We have used trapezoids to represent protected units. The rest of the notation is a simplified version of Buhr's notation, which Sanden calls Buhrgraphs [7], although Buhr them Structure Graphs. Package `Process_Plan` is effectively the same as Sanden's package `Proc_Plans`, except for the addition of

```
pragma Queuing_Policy (Priority_Queuing);
```

as the first line of the package specification. The rest of the design has changed. Packages `ASRS_Stand_Mgr`, `Workstations`, and `AGV_Mgr` use a reusable, unbounded, blocking queue component:

```
package Queue_Unbounded_Blocking is
   type Queue_Element  is tagged limited private;
   type Access_Element is access Queue_Element'Class;

   protected type Queue is
      procedure Put (Item : in Access_Element);
      -- Adds the item designated by "Item" to
      -- the tail of the queue

      entry Get (Item : out Access_Element);
```

```
    -- Removes the next element from the head of
    -- the queue and returns it in "Item".
    -- If the queue is empty, the caller is blocked
    -- until an element is added to the queue
```

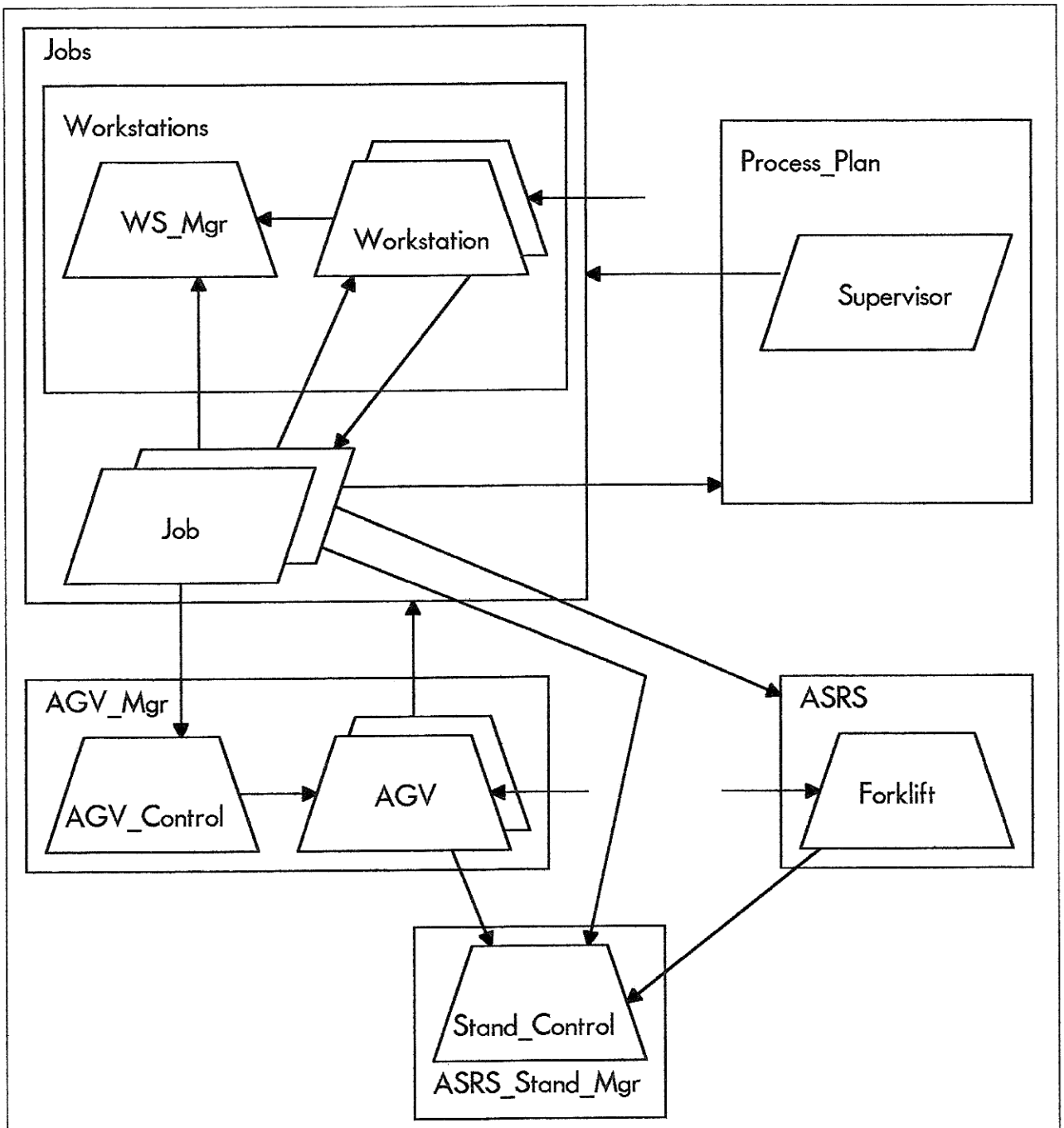

Figure 1. FMS Design

```
    function Empty return Boolean;
    -- Returns True if the queue is empty.
    -- Returns False otherwise
private -- Queue
    Head : Access_Element := null;
```

```
      Tail : Access_Element := null;
   end Queue;
private -- Queue_Unbounded_Blocking
   type Queue_Element is tagged record
      Next : Access_Element := null;
   end record;
end Queue_Unbounded_Blocking;
```

Carter presented this component in detail [5]. It uses inheritance to provide the component with the type of data to put on a queue. The examples below demonstrate its use.

7.2. Package ASRS_Stand_Mgr

Package `ASRS_Stand_Mgr` is a logical place to begin, because it is used by much of the rest of the system. The Ada-9X code for `ASRS_Stand_Mgr` is:

```
with Queue_Unbounded_Blocking;
package ASRS_Stand_Mgr is
   protected Stand_Control is
      entry Request (Stand : out Stand_Id); -- Reserve a stand
      procedure Release (Stand : in Stand_Id); -- Release a reserved stand
   private -- Stand_Control
      Queue : Queue_Unbounded_Blocking.Queue; -- A queue of Stand Ids
   end Stand_Control;
end ASRS_Stand_Mgr;

with Ada.Unchecked_Deallocation;
use Ada;
package body ASRS_Stand_Mgr is
   type Queue_Entry is new Queue_Unbounded_Blocking.Queue_Element with record
      Stand : Stand_Id;
   end record; -- Type of items Stand_Control will keep on its Queue

   procedure Free is new Unchecked_Deallocation
     (Object => Queue_Entry, Name => Queue_Unbounded_Blocking.Access_Element);

   protected body Stand_Control is
      entry Request (Stand : out Stand_Id) when not Queue.Empty is
         Ptr : Queue_Unbounded_Blocking.Access_Element;
      begin -- Request
         Queue.Get (Item => Ptr); -- Blocks caller if Queue is empty
         Stand := Ptr.Stand;
         Free (Ptr);
      end Request;

      procedure Release (Stand : in Stand_Id) is
         Ptr : Queue_Unbounded_Blocking.Access_Element :=
           new Queue_Entry'(Stand => Stand);
      begin -- Release
         Queue.Put (Item => Ptr);
      end Release;
   end Stand_Control;
begin -- ASRS_Stand_Mgr
   Fill : for Stand in Stand_Id loop -- Make all stands available
      Stand_Control.Release (Stand => Stand);
   end loop Fill;
end ASRS_Stand_Mgr;
```

`Stand_Control` maintains a queue of available ASRS stands, and provides one when requested. It is a straightforward replacement of a guardian task by a protected unit. `Stand_Control` basically hides the queue from callers.

7.3. The Forklift

Task `Fork` in package `ASRS` functions primarily as an interrupt handler. We can replace this task by a protected unit:

```
type State_Id is (Idle, Outbound, At_Bin_Out, Got, At_Stand_Out, Inbound,
                  At_Stand_In, Picked, At_Bin_In, Put, Done);
protected Forklift is
   entry Stand_To_Bin (Stand : in     Stand_Id; Bin : in Bin_Id);
   entry Bin_To_Stand (Stand :    out Stand_Id; Bin : in Bin_Id);
private -- Forklift
   procedure Fork_Done;
   pragma Attach_Handler (Fork_Done, 23);

   entry Finished_In  (Stand : in     Stand_Id; Bin : in Bin_Id);
   entry Finished_Out (Stand :    out Stand_Id; Bin : in Bin_Id);

   State         : State_Id := Idle;
   Current_Stand : Stand_Id;
   Current_Bin   : Bin_Id;
end Forklift;

protected body Forklift is
   entry Stand_To_Bin (Stand : in Stand_Id; Bin : in Bind_Id) when State = Idle
   is
      -- No declarations
   begin -- Stand_To_Bin
      Current_Stand := Stand; -- Save stand and bin in use
      Current_Bin := Bin;
      State := Inbound;
      To_Stand (Stand); -- Move physical forklift to stand
      requeue Finished_In; -- Make caller wait until movement is finished
   end Stand_To_Bin;

   entry Bin_To_Stand (Stand : out Stand_Id; Bin : in Bind_Id) when State = Idle
   is
      -- No declarations
   begin -- Bin_To_Stand
      -- Reserve a stand
      ASRS_Stand_Mgr.Stand_Control.Request (Stand => Current_Stand);
      Stand := Current_Stand; -- Save stand and bin in use
      Current_Bin := Bin;
      State := Outbound;
      To_Bin (Bin); -- Move physical forklift to bin
      requeue Finished_Out; -- Make caller wait until movement is finished
   end Bin_To_Stand;

   entry Finished_In (Stand : in Stand_Id; Bin : in Bind_Id) when State = Done is
      -- No declarations
   begin -- Finished
      State := Idle; -- Allow another job to obtain the forklift
   end Finished;

   entry Finished_Out (Stand : out Stand_Id; Bin : in Bind_Id)
      when State = Done
   is
      -- No declarations
   begin -- Finished
      State := Idle; -- Allow another job to obtain the forklift
   end Finished;

   procedure Fork_Done is
```

```
      -- No declarations
   begin -- Fork_Done
      case State is
      when Idle | Done => -- Shouldn't happen
         null;
      when Outbound => -- To_Bin completed for Bin_To_Stand
         State := At_Bin_Out;
         Bin_Get (Current_Bin);
      when At_Bin_Out => -- Bin_Get completed
         State := Got;
         To_Stand (Current_Stand);
      when Got => -- To_Stand completed for Bin_To_Stand
         State := At_Stand_Out;
         Place;
      when Inbound => -- To_Stand completed for Stand_To_Bin
         State := At_Stand_In;
         Pick;
      when At_Stand_In => -- Pick completed
         State := Picked;
         -- Stand is now free; release it
         ASRS_Stand_Mgr.Stand_Control.Release (Stand => Current_Stand);
         To_Bin (Current_Bin);
      when Picked => -- To_Bin completed for Stand_To_Bin
         State := At_Bin_In;
         Bin_Put (Current_Bin);
      when At_Stand_Out | At_Bin_In => -- Part at destination
         State := Done;
      end case;
   end Fork_Done;
end Forklift;
```

`State` records the last successful action. When the `Fork_Done` interrupt occurs, we know that the current action has successfully completed. `Fork_Done` updates `State` and starts the next action.

7.4. Package AGV_Mgr

Sanden used a number of tasks to implement AGVs. We can replace all these tasks by protected units, except for `AGV_Msgs`, which we can eliminate. `AGV_Msgs` might be waiting for an AGV to accept a call to `AGV_Done` when another interrupt occurs for another AGV. It is not clear what happens in Ada 83 if an interrupt occurs when the handling task is not ready to accept it. Using a protected type with discriminants allows us to have each AGV handle its own interrupts, eliminating this problem:

```
with Queue_Unbounded_Blocking, Ada.Interrupts, Ada.Unchecked_Deallocation;
use Ada;
package body AGV_Mgr is
   Num_AGVs : constant := ...;
   subtype AGV_Id is Positive range 1 .. Num_AGVs;

   type State_Id is (Idle, Moving, At_Output_Stand, Picked, At_Input_Stand,
                     Done);

   protected type AGV (Id : AGV_Id; Interrupt : Interrupts.Interrupt_Id) is
      entry Move (From : in Stand_Id; To : in Stand_Id);
   private -- AGV
      procedure AGV_Done;
      pragma Attach_Handler (AGV_Done, Interrupt);

      entry Finished (From : in Stand_Id; To : in Stand_Id);
```

```
   Output_Stand : Stand_Id;
   Input_Stand  : Stand_Id;
   State        : State_Id := Idle;
end AGV;
type AGV_Ptr is access AGV;

type Queue_Entry is new Queue_Unbounded_Blocking.Queue_Element with record
   AGV : AGV_Id := 1;
end record;

protected AGV_Control is
   entry Request (Ptr : out AGV_Ptr); -- Reserve an AGV & get a pointer to it
   procedure Available (Id : in AGV_Id); -- Make a reserved AGV available
private -- AGV_Control
   Queue : Queue_Unbounded_Blocking.Queue; -- Queue of available AGV Ids
end AGV_Control;

type AGV_Set is array (AGV_Id) of AGV_Ptr;
Owner : AGV_Set; -- Pointers to all AGVs

procedure Move (From : in Stand_Id; To : in Stand_Id) is
   Ptr : AGV_Ptr;
begin -- Move
   AGV_Control.Request (Ptr => Ptr); -- Reserve an available AGV
   Ptr.Move (From => From, To => To); -- Pass call to AGV
end Move;

procedure Free is new Unchecked_Deallocation
  (Object => Queue_Entry, Name => Queue_Unbounded_Blocking.Access_Element);

protected body AGV_Control is
   entry Request (Ptr : out AGV_Ptr) when not Queue.Empty is
      Queue_Ptr : Queue_Unbounded_Blocking.Access_Element;
   begin -- Request
      Queue.Get (Item => Queue_Ptr); -- Get an availabe AGV Id from Queue
      Ptr := Owner (Queue_Ptr.AGV); -- Extract the pointer to this AGV
      Free (Queue_Ptr);
   end Request;

   procedure Available (Id : in AGV_Id) is
      Ptr : Queue_Unbounded_Blocking.Access_Element :=
        new Queue_Entry'(AGV => Id);
   begin -- Available
      Queue.Put (Item => Ptr); -- Put this Id on the queue of available Ids
   end Available;
end AGV_Control;

protected body AGV is
   entry Move (From : in Stand_Id; To : Stand_Id) when State = Idle is
      -- No declarations
   begin -- Move
      Output_Stand := From; -- Record stands in use
      Input_Stand := To;
      State := Moving;
      -- Tell physical AGV to move to output stand
      requeue Finished; -- Make caller wait until movement is finished
   end Move;

   entry Finished (From : in Stand_Id; To : Stand_Id) when State = Done is
      -- No declarations
   begin -- Finished
```

```
            State := Idle; -- Make this AGV available to move another job
            AGV_Control.Available (Id => Id);
         end Finished;

         procedure AGV_Done is
            -- No declarations
         begin -- AGV_Done
            case State is
            when Idle | Done => -- Shouldn't happen
               null;
            when Moving => -- Arrived at output stand to pick up job
               State := At_Output_Stand;
               -- Tell physical AGV to pick
            when At_Output_Stand => -- Picked up the job
               State := Picked;
               -- Output stand is now free; allow other jobs to use it
               if Output_Stand in ASRS_Stand_Mgr.Stand_Id then
                  ASRS_Stand_Mgr.Stand_Control.Release (Stand => Output_Stand);
               else
                  Jobs.Release_Output_Stand (Output_Stand);
               end if;
               -- Tell physical AGV to move to input stand
            when Picked => -- Arrived at input stand to put down job
               State := At_Input_Stand;
               -- Tell physical AGV to place
            when At_Input_Stand => -- Put down the job
               State := Done; -- Finished, release the caller
            end case;
         end AGV_Done;
      end AGV;
   begin -- AGV_Mgr
      Create : for Id in Owner'Range loop -- Create the AGVs & make them available
         Owner (Id) := new AGV'(Id => Id, Interrupt => Get_Interrupt_Id (Id) );
         AGV_Control.Available (Id => Id);
      end loop Create;
   end AGV_Mgr;
```

When a `Job` task needs an AGV, it calls `AGV_Mgr.Move`, which obtains a pointer to an AGV from `AGV_Control` and invokes its `Move` entry. When the AGV has finished, it tells `AGV_Control` that it is again available. The use of a queue is similar to `Stand_Control`, and the interrupt handler `AGV_Done` is similar to `Fork_Done`.

7.5. Package Workstations

Package `Workstations` changes the most. Each workstation will be represented by a protected unit, which will handle all its operations. As with the interrupts for AGVs, Sanden's `WS_Msgs` task could be a serious bottleneck in the system when several workstations completed an action at about the same time. With protected units, each workstation can handle its own stream of messages. In addition, the workstation can release its own input stand, eliminating the need for the job task to have an `Input_Stand_Clear` entry or to call `Release_Input_Stand`.

```
   package Workstations is
      protected type Workstation (Kind : WS_Kind; Id : WS_Id) is
         procedure On_Input_Stand (Job : in Job_Ptr);
         procedure WS_Msg (Msg : in Msg_Info);
      private -- Workstation
         Output_Job : Job_Ptr; -- Job on output stand
         Tool_Job   : Job_Ptr; -- Job in the tool
         Input_Job  : Job_Ptr; -- Job on input stand
```

```
    end Workstation;

    type WS_Ptr is access Workstation;

    protected WS_Mgr is
       entry Request (Kind : in WS_Kind; Id : out WS_Id; Ptr : out WS_Ptr);
    end WS_Mgr;
end Workstations;

package body Workstations is
    protected type Id_Queue is
       entry Request (Kind : in WS_Kind; Id : out WS_Id; Ptr : out WS_Ptr);
       procedure Available (Id : in WS_Id);
    private -- Id_Queue
       Queue : Queue_Unbounded_Blocking.Queue;
    end Id_Queue;

    type Queue_Set is array (WS_Kind) of Id_Queue;
    type Ptr_Set is array (WS_Kind, WS_Id) of WS_Ptr;

    Kind_Mgr    : Queue_Set;
    WS          : Ptr_Set;

    protected body WS_Mgr is
       entry Request (Kind : in WS_Kind; Id : out WS_Id; Ptr : out WS_Ptr)
         when True
       is
          -- No declarations
       begin -- Request
          -- Just direct the request to the correct kind of workstation
          requeue Kind_Mgr (Kind).Request;
       end Request;
    end WS_Mgr;

    protected body Workstation is
       procedure On_Input_Stand (Job : in Job_Ptr) is
          -- No declarations
       begin -- On_Input_Stand
          Input_Job := Job; -- Record the job
       end On_Input_Stand;

       procedure WS_Msg (Msg : in Msg_Info) is
          -- No declarations
       begin -- WS_Msg
          case Msg.Event is
          when Input_Stand_Clear => -- Job moved from input stand into tool
             Tool_Job := Input_Job; -- Move job through workstation
             Input_Job := null;
             -- Workstation's input stand is now available
             Kind_Mgr (Kind).Available (Id => Id);
          when Clear_Output_Stand => -- Job in tool finished; need output stand
             if Output_Job /= null then
                select
                   Output_Job.Clear_Output_Stand; -- Bump job
                else
                   null; -- Job already acquired next workstation
                end select;
             end if;
          when On_Output_Stand => -- Job moved from tool onto output stand
             Output_Job := Tool_Job; -- Move job through workstation
             Tool_Job := null;
```

```
                Output_Job.On_Output_Stand (Aborted => Msg.Aborted); -- Notify job
            end case;
        end WS_Msg;
    end Workstation;

    -- body of Id_Queue ...
begin -- Workstations
    All_Kinds : for Kind in WS_Kind loop -- Create all workstations
        All_Ids : for Id in 1 .. Num_WSs (Kind) loop
            WS (Kind, Id) := new Workstation'(Kind => Kind, Id => Id);
            Kind_Mgr (Kind).Available (Id => Id);
        end loop All_Ids;
    end loop All_Kinds;
end Workstations;
```

Protected unit `WS_Mgr` protects no data, and its entry `Request` has a guard which is always true. `Request` is an entry, rather than a procedure, to allow it to requeue its calls on the manager for the desired kind of workstation. `WS_Mgr` exists only to hide the collection of managers from the caller.

When a job obtains a workstation, it gets a pointer to a `Workstation`. When the AGV delivers the job to the workstation's input stand, the job calls the workstation's `On_Input_Stand` procedure with a pointer to the job task. The workstation then tracks the job's progress through the tool by moving the job pointer in response to messages from the workstation's microprocessor. When the workstation has finished with the job and moved it to the output stand, the workstation calls the job's `On_Output_Stand` entry. The job can then try to obtain a workstation for its next step.

Sanden made the call to `Clear_Output_Stand` a timed entry call, concerned that a job might miss the call if it were suspended between queuing for the workstation and accepting the call. The asynchronous select statement removes that opportunity for suspension. In any case, physically moving a part into the workstation tool and machining it takes seconds, if not minutes, while a job task takes only milliseconds (at most) to set up the asynchronous select statement after being told it is on the workstation's output stand. If `Output_Job` is not ready to accept a call to `Clear_Output_Stand` immediately, we can be confident that it has already obtained its next workstation. This allows us to change Sanden's timed entry call into a conditional entry call.

7.6. Task Job

Finally, we can rewrite task `Job`. A new field needs to be added to the `Q_Elt_Type` record:

```
Task_Id : Ada.Task_Identification.Task_Id :=
  Ada.Task_Identification.Null_Task_Id;
```

This is needed to allow `Supervisor` to change a job's priority. `Job` then looks like:

```
task type Job (Q : Q_Ptr_Type) is
    entry On_Output_Stand (Aborted : in Boolean);
    entry Clear_Output_Stand;
end Job;

task body Job is
    Input_Stand  : Stand_Id;
    Output_Stand : Stand_Id;
    WS           : WS_Id;
    WS_Ptr       : Workstations.WS_Ptr;
    Step         : Step_Record;
begin -- Job
```

```
        Q.Task_Id := Ada.Task_Identification.Current_Task;
        Step := Get_Step_Info (Q.Plan, Q.Step); -- Get info for 1st step

        Completion : loop
            -- Job is in ASRS bin
            Workstations.WS_Mgr.Request
              (Kind => Step.WS_Kind, Id => WS, Ptr => WS_Ptr);
            -- Workstation has been assigned. Move from bin to stand

            ASRS.Bin_To_Stand (Output_Stand, Q.Bin);
            Q.In_ASRS := False;

            Floor : loop
                -- Job is on output stand. Move to input stand.
                Input_Stand := In_Stand (Step.WS_Kind, WS);
                AGV_Mgr.Move (From => Output_Stand, To => Input_Stand);
                WS_Ptr.On_Input_Stand (Job => Q.Task_Ptr);
                -- Job placed on input stand. WS informed.

                accept On_Output_Stand (Aborted : in Boolean) do
                    -- WS finished with job. Job is on output stand, possibly aborted.
                    Q.Abt := Aborted;
                end On_Output_Stand;
                Output_Stand := Out_Stand (Step.WS_Kind, WS);
                exit Floor when Q.Abt; -- Exit if aborted

                Q.Step := Q.Step + 1; -- Get info for next step
                Step := Get_Step_Info (Q.Plan, Q.Step);

                Q.Done := Step.Done;
                exit Floor When Q.Done; -- Exit if finished

                select -- Get workstation for next step, or be bumped back to ASRS
                    Workstations.WS_Mgr.Request -- Get workstation
                      (Kind => Step.WS_Kind, Id => WS, Ptr => WS_Ptr);
                then abort
                    accept Clear_Output_Stand; -- Be bumped
                    exit Floor; -- Exit if bumped
                end select;
            end loop Floor;

            -- Move job to ASRS
            ASRS_Stand_Mgr.Stand_Control.Request (Input_Stand);
            AGV_Mgr.Move (From => Output_Stand, To => Input_Stand);
            ASRS.Stand_To_Bin (Input_Stand, Q.Bin);
            Q.In_ASRS := True;

            exit Completion when Q.Abt or Q.Done; --Exit if finished
        end loop Completion;

        Q.Task_Ptr := null;
        Q.Task_Id := Ada.Task_Identification.Null_Task_Id;
    end Job;
```

Jobs are dynamically created by the supervisor and are given their queue pointers then. Jobs arrange to be moved from place to place and spend the rest of their time waiting for resources such as AGVs or for the movement or the processing to finish. These are the qualities Sanden calls "queueability" and "reschedulability," which make jobs attractive subjects for the software.

8. Conclusion

ELM combines object- and processed-based design concepts, resulting in designs for reactive software which closely model the problem being solved. These designs are inherently concurrent and modular, making use of information hiding and requiring exclusive access to shared resources. As the FMS example demonstrates, using Ada-9X features for systems and real-time programming to implement ELM designs not only reduces the execution overhead of the software, but also results in simpler code.

References

1. Sanden, B., *Software Systems Construction with Examples in Ada*, Prentice-Hall, 1994
2. Anderson, C., Presentation made at the 1993 Washington Ada Symposium
3. Carter, J., "The Form of Reusable Ada Components for Concurrent Use," *Ada Letters*, 1990 Jan/Feb
4. Carter, J., "Concurrent Reusable Abstract Data Types," *Ada Letters*, 1991 Jan/Feb
5. Carter, J., "Ada 9X Reusable Components," *Ada Letters*, 1992 Mar/Apr
6. Ada 9X Mapping/Revision Team, *Ada 9X Reference Manual*, Draft Version 4.0, Intermetrics, Inc., 1993
7. Buhr, R., *System Design with Ada*, Prentice-Hall, 1984